

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Лабораторная работа №3
по дисциплине
«Методы тестирования программного обеспечения»

Студент,

группы 5130201/30101

_____ Мелещенко С. И.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Описание тестируемой программы	5
3 Исходный код программы	7
4 Статический анализ кода приложения с помощью PVS-Studio	10
5 Работа с PVS-Studio	14
6 Исправленный код программы	20
Заключение	24

Введение

В течение многих лет большинство программистов исходили из предположения, что программы пишутся исключительно для машинной обработки и не предназначены для чтения человеком, а единственным способом из тестирования является выполнение программного кода в компьютере. Ситуация стала меняться в начале 1970-х годов благодаря усилиям тех разработчиков, которые впервые осознали ценность просмотра кода как одного из элементов процесса тестирования и отладки программ.

Статическое тестирование - процесс анализа программного кода без его выполнения. Целью статического тестирования является выявление недочетов на ранних стадиях разработки, что позволяет сэкономить время и ресурсы.

В данной работе использовался статический анализатор PVS-Studio как SAST-решение. Он выявляет не только опечатки, мёртвый код и другие ошибки, но и потенциальные уязвимости (Static Application Security Testing). Отображает предупреждения на Common Weakness Enumeration, SEI CERT Coding Standards, а также поддерживает стандарт MISRA.

В данной работе рассматривается программа умножения матриц с реализацией двух алгоритмов: классического и оптимизированного (с блочным умножением).

1 Постановка задачи

Цель работы — проанализировать код программы умножения матриц с целью улучшения качества кода и выявления потенциальных ошибок. Необходимо изучить инструмент статического анализа PVS-Studio и провести анализ Java-программы.

Задачи:

1. Изучить метод статического анализа кода и инструмент PVS-Studio.
2. Установить и настроить PVS-Studio для работы с Java-проектами в среде VS Code.
3. Провести статический анализ кода при помощи PVS-Studio.
4. Проанализировать полученные предупреждения.
5. Выявить потенциальные ошибки и недочеты в коде.
6. Исправить обнаруженные проблемы.

Дано: программный код на Java (умножение матриц с реализацией классического и оптимизированного алгоритмов).

Программа реализует следующие функции:

1. Ввод размерностей матриц с проверкой корректности.
2. Проверка возможности перемножения матриц.
3. Заполнение матриц случайными значениями.
4. Умножение матриц классическим алгоритмом ($O(n^3)$).
5. Умножение матриц оптимизированным блочным алгоритмом для улучшения кэш-производительности.
6. Замер времени выполнения 10 запусков каждого алгоритма с выводом среднего времени.

2 Описание тестируемой программы

Название: Калькулятор для умножения матриц с измерением и выводом производительности.

Дано.

1. Консольный ввод пользователя.
2. Четыре положительных целых числа, задающих размеры двух матриц:
 - Количество строк первой матрицы.
 - Количество столбцов первой матрицы.
 - Количество строк второй матрицы.
 - Количество столбцов второй матрицы.
3. Программа автоматически заполняет матрицы случайными значениями типа `double`.

Требуется.

1. Проверить возможность умножения матриц (количество столбцов первой матрицы должно быть равно количеству строк второй).
2. Выполнить умножение матриц стандартным методом (тройным вложенным циклом) и измерить время выполнения (10 прогонов).
3. Выполнить умножение матриц оптимизированным методом (блочным умножением с размером блока 64) и измерить время выполнения (10 прогонов).
4. Вывести среднее время выполнения для каждого метода.

Ограничения.

1. Размеры матриц должны быть положительными целыми числами.
2. Умножение возможно только если количество столбцов первой матрицы равно количеству строк второй.
3. Матрицы заполняются случайными числами в диапазоне $[0, 1)$.
4. Время измеряется в миллисекундах.

Спецификация

Входные данные	Ожидаемый результат	Комментарий
2, 2, 2, 2	Успешное умножение. Вывод времени выполнения для обоих методов.	Минимальные квадратные матрицы (2x2).
3, 4, 4, 5	Успешное умножение. Вывод времени выполнения.	Прямоугольные матрицы (3x4 и 4x5).
3, 2, 3, 2	Ошибка: "Матрицы такой размерности нельзя перемножить!"	Несовместимые размеры (3x2 и 3x2).
0, 5, 5, 5	Ошибка: "Ошибка! Введите положительное число!"	Некорректный размер (ноль строк).
5, -2, 5, 5	Ошибка: "Ошибка! Введите положительное число!"	Некорректный размер (отрицательные столбцы).
abc, 5, 5, 5	Ошибка: "Ошибка! Введите целое число!"	Нечисловой ввод для размера.
5, 5, 5.5, 5	Ошибка: "Ошибка! Введите целое число!"	Дробный ввод для размера.
1, 1000, 1000, 1	Успешное умножение. Результат - матрица 1x1 (скалярное произведение).	Матрица-строка и матрица-столбец.
50, 100, 100, 50	Успешное умножение. Вывод среднего времени за 10 прогонов для каждого метода.	Типичный случай умножения прямоугольных матриц.
200, 200, 200, 200	Успешное умножение. Явное преимущество блочного метода по времени.	Большие матрицы для демонстрации оптимизации.

3 Исходный код программы

Листинг 1. Код программы

```
1 public class MatrixMult {
2     public static double[][] multiply(double[][] firstMatrix ,
3         double[][] secondMatrix) {
4         int firstRows = firstMatrix.length;
5         int firstColumns = firstMatrix[0].length;
6         int secondRows = secondMatrix.length;
7         int secondColumns = secondMatrix[0].length;
8         double[][] result = new double[firstRows][secondColumns
9             ];
10        for (int i = 0; i < firstRows; i++) {
11            for (int j = 0; j < firstColumns; j++) {
12                for (int k = 0; k < secondColumns; k++) {
13                    result[i][k] += firstMatrix[i][j] *
14                        secondMatrix[j][k];
15                }
16            }
17        }
18        return result;
19    }
20
21    public static int readPositiveInt(Scanner scanner, String
22        message) {
23        while (true) {
24            System.out.print(message);
25            if (scanner.hasNextInt()) {
26                int value = scanner.nextInt();
27                if (value > 0) {
28                    return value;
29                } else { System.out.println("Ошибка! Введите пол
30                    ожительное число!");
31                }
32            } else { System.out.println("Ошибка! Введите целое ч
33                исло!");
34            }
35            scanner.next(); //очищаем некорректный ввод
36        }
37    }
38
39    public static double[][] multiplyOptimized(double[][]
40        firstMatrix, double[][] secondMatrix) {
41        int rows1 = firstMatrix.length;
42        int cols1 = firstMatrix[0].length;
43        int cols2 = secondMatrix[0].length;
44        double[][] result = new double[rows1][cols2];
45        int blockSize = 64;
```

```

37     for (int blockI = 0; blockI < rows1; blockI += blockSize
38         ) {
39         for (int blockJ = 0; blockJ < cols2; blockJ +=
40             blockSize) {
41             for (int blockK = 0; blockK < cols1; blockK +=
42                 blockSize) {
43                 for (int i = blockI; i < Math.min(blockI +
44                     blockSize, rows1); i++) {
45                     for (int k = blockK; k < Math.min(blockK
46                         + blockSize, cols1); k++) {
47                         for (int j = blockJ; j < Math.min(
48                             blockJ + blockSize, cols2); j++)
49                             {
50                                 result[i][j] += firstMatrix[i][k
51                                     ] * secondMatrix[k][j];
52                             }
53                         }
54                     }
55                 }
56             }
57         }
58     }
59     return result;
60 }
61
62 public static void main(String[] args) {
63     Scanner scanner = new Scanner(System.in);
64     int firstRows = readPositiveInt(scanner, "Введите количе
65     ство строк для первой матрицы: ");
66     int firstColumns = readPositiveInt(scanner, "Введите кол
67     ичество столбцов для первой матрицы: ");
68     int secondRows = readPositiveInt(scanner, "Введите колич
69     ество строк для второй матрицы: ");
70     int secondColumns = readPositiveInt(scanner, "Введите ко
71     личество столбцов для второй матрицы: ");
72     if (firstColumns != secondRows) {
73         System.out.println("Ошибка! Матрицы такой размерност
74         и нельзя перемножить! Столбцов в первой матрице д
75         олжно быть столько же, сколько строк во второй!");
76     }
77     scanner.close();
78     return;
79 }
80 double[][] matrix1 = new double[firstRows][firstColumns
81 ];
82 double[][] matrix2 = new double[secondRows][
83     secondColumns];
84 for (int i = 0; i < firstRows; i++) {
85     for (int j = 0; j < firstColumns; j++) {
86         matrix1[i][j] = Math.random();

```



```

68     }
69 }
70 for (int i = 0; i < secondRows; i++) {
71     for (int j = 0; j < secondColumns; j++) {
72         matrix2[i][j] = Math.random();
73     }
74 }
75 int runs = 10;
76 long totalTime = 0;
77 for (int run = 0; run < runs; run++) {
78     long startTime = System.nanoTime();
79     double[][] result = multiply(matrix1, matrix2);
80     long endTime = System.nanoTime();
81     long duration = (endTime - startTime) / 1_000_000;
82     totalTime += duration;
83     System.out.println((run + 1) + ": " + duration + " м
84         c");
85 }
86 System.out.println("Среднее время за 10 перемножений: "
87     + (totalTime / runs) + " мс");
88 totalTime = 0;
89 for (int run = 0; run < runs; run++) {
90     long startTime = System.nanoTime();
91     double[][] result = multiplyOptimized(matrix1,
92         matrix2);
93     long endTime = System.nanoTime();
94     long duration = (endTime - startTime) / 1_000_000;
95     totalTime += duration;
96     System.out.println((run + 1) + ": " + duration + " м
97         c");
98 }
99 System.out.println("Среднее время за 10 оптимизированных
100     перемножений: " + (totalTime / runs) + " мс");
101 scanner.close();
102 }

```

4 Статический анализ кода приложения с помощью PVS-Studio

PVS-Studio — проприетарный статический анализатор кода для программ, написанных на C, C++, C++/CLI, C++/CX, C и на Java. Разрабатывается ООО «ПВС».

Приложение может интегрироваться со средами разработки Microsoft Visual Studio, Visual Studio Code, IntelliJ IDEA, Rider, CLion, SonarQube, DefectDojo, Jenkins, TeamCity, IncrediBuild, Qt Creator.

Анализатор PVS-Studio разрабатывается с учётом требований, предъявляемых к статическим анализаторам в ГОСТ Р 71207-2024 (Статический анализ программного обеспечения. Общие требования).

Виды анализа (п. 7.4.)	С и C++	C#	Java
Внутрипроцедурный анализ потоков данных и управления	✓	✓	✓
Межпроцедурный и межмодульный контекстно-чувствительный анализ потока данных	✓	✓	✓
Чувствительный к путям выполнения анализ потоков данных и управления	✓	✓	✓
Межпроцедурный и межмодульный контекстно-чувствительный анализ помеченных данных	✓	✓	✓*
Анализ программы на синтаксическом уровне	✓	✓	✓

Рис. 1. Виды анализа, которые должен реализовывать статический анализатор кода согласно ГОСТ Р 71207–2024 (п. 7.4.). Данные приведены для анализатора PVS-Studio на конец 2024 года.

Основные возможности PVS-Studio

PVS-Studio предоставляет широкий набор функций для анализа кода, которые помогают разработчикам улучшить качество программного обеспечения.

1. Анализ и поиск ошибок - PVS-Studio позволяет выявлять широкий спектр ошибок, включая:

- Опечатки в коде: Например, случайное использование `=` вместо `==`, или ошибки в логических выражениях.

- Неправильное использование переменных: Использование неинициализированных переменных, утечки памяти, доступ к освобождённой памяти.

- Ошибки многопоточности: Проблемы с синхронизацией, гонки данных, некорректное использование потоков.

- Неопределённое поведение: Код, который может вести себя непредсказуемо в зависимости от компилятора или платформы.

- Потенциальные уязвимости: Выявление мест в коде, которые могут быть эксплуатированы злоумышленниками (например, SQL-инъекции, переполнение буфера).

2. Анализ многопоточного кода.

3. Удобные отчёты - PVS-Studio генерирует отчёты с подробным описанием найденных ошибок: ошибки разделяются на категории (например, критические, предупреждения, улучшения), инструмент предоставляет объяснение каждой ошибки, примеры неправильного и правильного кода, возможность фильтровать предупреждения по важности, типу ошибок или файлам, поддержка различных форматов.

4. Проверка соответствия стандартам.

Виды ошибок

1. Опечатки — одна из наиболее распространённых причин ошибок. PVS-Studio умеет находить такие проблемы.

2. Использование неинициализированных переменных - инструмент обнаруживает случаи, когда переменная используется до её инициализации.

3. Утечки памяти - PVS-Studio помогает выявлять утечки памяти.

4. Доступ к невалидной памяти. Инструмент находит попытки обращения к освобождённой памяти, выход за пределы массива и другие

подобные ошибки.

5. Ошибки многопоточности. PVS-Studio помогает выявлять проблемы многопоточности:

- Гонки данных (data races).
- Deadlock'и.
- Неправильное использование примитивов синхронизации.

6. Логические ошибки.

- Условия, которые всегда истинны или ложны.
- Некорректные циклы.
- Ошибки в выражениях.

7. Неопределённое поведение. Инструмент выявляет код, который может вести себя по-разному в зависимости от компилятора, платформы или окружения.

8. Неиспользуемый или избыточный код. Инструмент находит:

- Неиспользуемые переменные, функции и параметры.
- Лишние вызовы функций.
- Код, который не влияет на результат выполнения программы.

Диагностические правила

1. General (GA) - диагностики общего плана. Основной набор диагностических правил PVS-Studio.

2. Optimization (OP) - диагностики микрооптимизации. Указания по повышению эффективности и безопасности кода.

3. 64-bit (64) - диагностики, позволяющие выявлять специфические ошибки, связанные с разработкой 64-битных приложений, а также переносом кода с 32- битной на 64-битную платформу.

4. Customers' Specific (CS) - узкоспециализированные диагностики, разработанные по просьбам пользователей.

5. MISRA - диагностики, разработанные в соответствии со стандартом MISRA (Motor Industry Software Reliability Association).

Уровни предупреждений

1. High - включает в себя предупреждения с максимальным уровнем достоверности. Такие предупреждения часто указывают на ошибки, требующие немедленного исправления.

2. Medium - содержит менее достоверные предупреждения, на которые все же стоит обратить пристальное внимание.

3. Low - предупреждения с минимальным уровнем достоверности, указывающие на несущественные неточности в коде. Среди таких предупреждений обычно велик процент ложных срабатываний.

4. Fails - внутренние сообщения анализатора, информирующие о возникновении каких-то проблем при работе. В группу Fails попадают сообщения об ошибках анализатора (например, сообщения с кодами V001, V003 и т.п.), а также любой необработанный вывод вспомогательных программ, используемых самим анализатором во время анализа (препроцессор, командный процессор cmd), выдаваемый ими в stdout/stderr. Например, в группу Fails может попасть сообщение препроцессора об ошибках препроцессирования, доступа к файлам (файл не существует или заблокирован антивирусом) и т.п.

5 Работа с PVS-Studio

Для проведения статического анализа кода необходимо было установить и настроить инструмент PVS-Studio. Работа выполнялась в операционной системе Windows 11, среда разработки — Visual Studio Code.

Первым шагом была загрузка дистрибутива PVS-Studio с официального сайта <https://pvs-studio.com/>. Был выбран дистрибутив для Windows, включающий поддержку Java.

При установке в окне выбора компонентов были отмечены следующие опции:

- Java analyzer core — ядро анализатора для Java;
- Integration with Visual Studio Code — интеграция с VS Code;
- Core — основные утилиты анализатора.

Расширение PVS-Studio для VS Code было установлено через маркетплейс расширений.

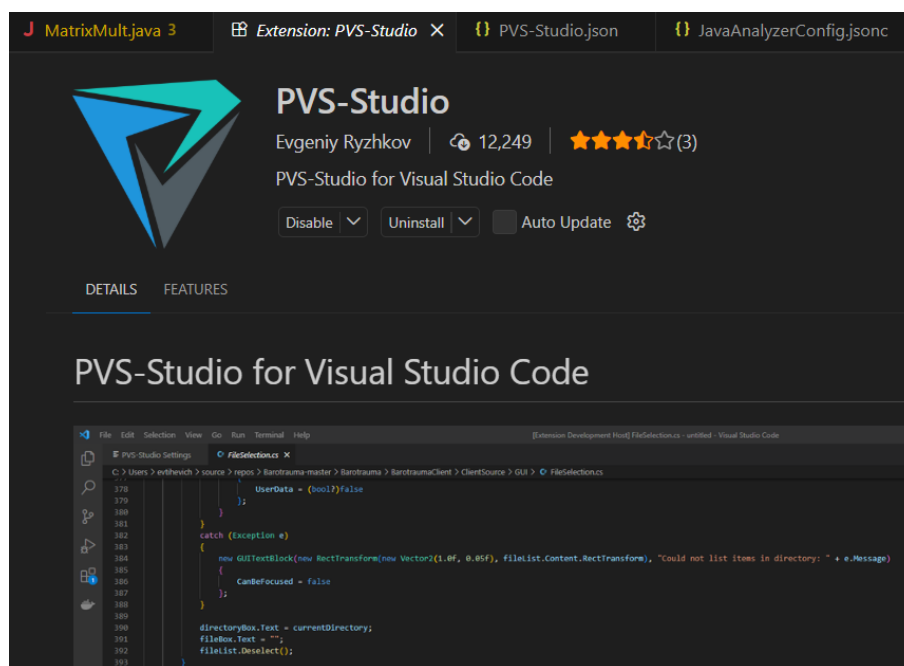


Рис. 2. PVS-Studio

После установки в нижней панели VS Code появилась вкладка PVS-Studio, содержащая интерфейс для управления анализом.

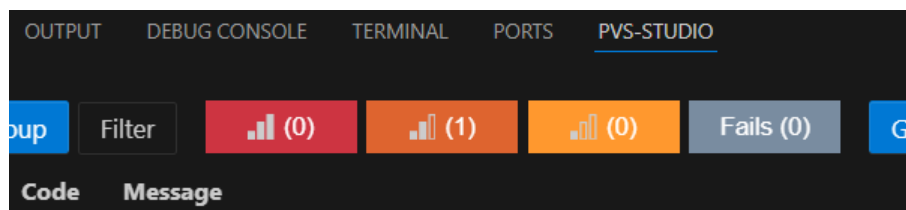


Рис. 3. PVS-Studio

Для указания пути к JDK в настройках PVS-Studio был создан файл конфигурации `JavaAnalyzerConfig.jsonc` в папке `.PVS-Studio` проекта:

Листинг 2. Настройки `JavaAnalyzerConfig.jsonc`

```

1 {
2   "java-path": "C:\\Program Files\\Java\\jdk-25.0.2\\bin\\java.exe",
3   "compatibility": true,
4   "source-java": 8,
5   "target-java": 8,
6   "logging": "INFO"
7 }
```

В начало файла `MatrixMult.java` добавлен комментарий, необходимый для студенческой лицензии:

```

1 // This is a personal academic project. Dear PVS-Studio, please
   check it.
2 // PVS-Studio Static Code Analyzer for C, C++, C#, and Java:
   https://pvs-studio.com
```

Анализ был запущен следующим образом:

1. Открыта вкладка PVS-Studio в нижней панели VS Code.
2. Нажата кнопка Start analysis (зеленый треугольник).

Диагностические возможности PVS-Studio, применённые при анализе

PVS-Studio для Java включает более 450 диагностических правил, разделённых на несколько групп. В ходе анализа нашей программы были активированы все правила общего назначения (GA), что позволило проверить код по следующим направлениям (каждое направление содержит десятки конкретных проверок):

а. Синтаксическая корректность и стиль кода

- Пропущенные точки с запятой – выявление мест, где оператор не завершён знаком ;.

- Незакрытые скобки – проверка баланса круглых, фигурных и квадратных скобок.

- Некорректное использование операторов – например, `=` вместо `==` в условных выражениях.

- Избыточные или недостижимые конструкции – код, который никогда не выполняется (dead code).

- Нарушения соглашений об именовании – проверка соответствия стиля (CamelCase для классов, lowerCamelCase для методов и переменных).

- Некорректные модификаторы доступа – например, `public static` для поля, которое должно быть приватным.

б. Анализ неиспользуемого и избыточного кода

- Неиспользуемые локальные переменные – объявленные, но нигде не читаемые и не изменяемые (обнаружена переменная `secondRows`).

- Неиспользуемые параметры методов – параметры, которые никогда не используются внутри метода.

- Неиспользуемые поля класса – поля, к которым нет обращений.

- Неиспользуемые методы – методы, которые не вызываются ни из одной точки программы.

- Мёртвые присваивания – переменной присваивается значение, которое затем нигде не используется.

- Избыточные проверки – условия, которые всегда истинны или всегда ложны.

в. Ошибки работы с массивами и коллекциями

- Выход за границы массива – индексы, потенциально выходящие за пределы длины массива.

- Некорректное использование `length` vs `length()` – путаница между полем массива и методом строки.

- Сравнение массивов через `==` – вместо `Arrays.equals()`.

- Использование `ArrayList` без указания начальной ёмкости – потенциальное снижение производительности.

- Потенциальные `NullPointerException` при обращении к элементам коллекции.

г. Проблемы с типами данных и преобразованиями

- Потеря точности при преобразовании типов – например, double -> int без явного приведения.

- Сравнение чисел с плавающей точкой через == – риск ошибок из-за погрешностей.

- Некорректное использование float и double – например, использование float для финансовых расчётов.

- Переполнение целочисленных типов – арифметические операции, которые могут выйти за пределы диапазона.

- Сужение примитивных типов без явного приведения – потенциальная потеря данных.

д. Логические ошибки в условиях и циклах

- Условия, которые всегда истинны или всегда ложны – сигнал о возможной опечатке.

- Бесконечные циклы – отсутствие изменения переменной условия.

- Циклы с неверным шагом – например, for (int i=0; i<10; i-).

- Сложные условия с неправильным приоритетом операторов – рекомендуется использовать скобки.

- Дублирование условий – одинаковые проверки в if-else цепочке.

е. Работа с ресурсами и исключениями

- Незакрытые ресурсы – Scanner, файловые потоки, сетевые соединения, не закрытые в finally или try-with-resources.

- Перехват исключений типа Exception – слишком широкий перехват, скрывающий специфические ошибки.

- Выброс null – использование throw null;

- Пустые блоки catch – подавление исключений без обработки.

ж. Безопасность (SAST)

- SQL-инъекции – конкатенация строк для формирования SQL-запросов.

- Небезопасное использование System.out и System.err – логирование конфиденциальной информации.

- Потенциальные уязвимости ввода-вывода – невалидация пользовательского ввода.

- Использование небезопасных функций – например, Runtime.exec()

без санитизации аргументов.

з. Микрооптимизация и производительность

- Использование String вместо StringBuilder в циклах – избыточное создание объектов.

- Медленные операции над строками – String.toLowerCase() в циклах.

- Неэффективные обращения к коллекциям – поиск в LinkedList по индексу.

- Избыточная синхронизация – блокировки там, где они не нужны.

и. Многопоточность (хотя программа однопоточная, диагностики также проверяют)

- Гонки данных (data races) – доступ к общим переменным без синхронизации.

- Некорректное использование volatile – недостаточность для атомарных операций.

- Deadlock-потенциал – порядок захвата блокировок.

- Запуск потока в конструкторе – утечка ссылки this.

к. Соответствие стандартам

- MISRA Java – правила безопасного программирования (например, избегание goto, ограничения на использование операторов).

- SEI CERT Coding Standards – рекомендации по безопасности (например, правила работы с памятью, исключениями).

- CWE (Common Weakness Enumeration) – сопоставление предупреждений с общепризнанными типами уязвимостей.

Общее количество активных диагностических правил – **452** (из них в группе GA – 382 правила, в группе OP – 48, в группе 64 – 14, остальные – специализированные).

Количество предупреждений, выданных анализатором – 1 (уровень Low, код V6021).

Количество файлов, проанализированных – 1 (MatrixMult.java).

Время анализа – примерно 10 секунд.

Таким образом, даже одно предупреждение было получено после проверки кода по сотням диагностических правил.

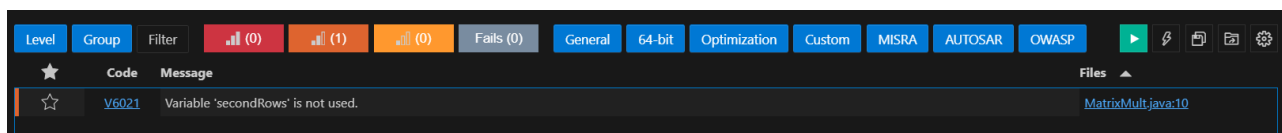


Рис. 4. Результат анализа

1. Low (Уровень достоверности) - низкий уровень достоверности (Low certainty).

2. V6021 (Код диагностики) - уникальный идентификатор диагностического правила в PVS-Studio. Что проверяет это правило: анализатор находит переменные, которые были объявлены, инициализированы, но нигде не используются в дальнейшем коде.

3. "Variable 'secondRows' is not used"(Сообщение) - "Переменная 'secondRows' не используется".

4. "MatrixMult.java"(Имя файла) - файл, в котором обнаружена проблема.

5. "10"(Номер строки) - строка в файле, где объявлена проблемная переменная.

Полученное предупреждение V6021 (неиспользуемая переменная) имело низкий уровень достоверности (Low), однако даже такие предупреждения были проанализированы и исправлены для повышения качества кода. После внесения исправлений повторный запуск анализатора подтвердил отсутствие аналогичных проблем.

6 Исправленный код программы

Для исправления ошибки было закомментировано (убрано) объявление переменной.

Листинг 3. Код программы

```
1 public class MatrixMult {
2     public static double[][] multiply(double[][] firstMatrix ,
3         double[][] secondMatrix) {
4         int firstRows = firstMatrix.length;
5         int firstColumns = firstMatrix[0].length;
6         //int secondRows = secondMatrix.length; — убираем (заком
7             ментируем)
8         int secondColumns = secondMatrix[0].length;
9         double[][] result = new double[firstRows][secondColumns
10             ];
11         for (int i = 0; i < firstRows; i++) {
12             for (int j = 0; j < firstColumns; j++) {
13                 for (int k = 0; k < secondColumns; k++) {
14                     result[i][k] += firstMatrix[i][j] *
15                         secondMatrix[j][k];
16                 }
17             }
18         }
19         return result;
20     }
21 }
22
23 public static int readPositiveInt(Scanner scanner, String
24     message) {
25     while (true) {
26         System.out.print(message);
27         if (scanner.hasNextInt()) {
28             int value = scanner.nextInt();
29             if (value > 0) {
30                 return value;
31             } else { System.out.println("Ошибка! Введите пол
32                 ожительное число!");
33             }
34         } else { System.out.println("Ошибка! Введите целое ч
35             исло!");
36             scanner.next(); //очищаем некорректный ввод
37         }
38     }
39 }
40
41 public static double[][] multiplyOptimized(double[][]
42     firstMatrix, double[][] secondMatrix) {
43     int rows1 = firstMatrix.length;
44     int cols1 = firstMatrix[0].length;
45     int cols2 = secondMatrix[0].length;
```

```

35 double[][] result = new double[rows1][cols2];
36 int blockSize = 64;
37 for (int blockI = 0; blockI < rows1; blockI += blockSize
38 ) {
39     for (int blockJ = 0; blockJ < cols2; blockJ +=
40         blockSize) {
41         for (int blockK = 0; blockK < cols1; blockK +=
42             blockSize) {
43             for (int i = blockI; i < Math.min(blockI +
44                 blockSize, rows1); i++) {
45                 for (int k = blockK; k < Math.min(blockK
46                     + blockSize, cols1); k++) {
47                     for (int j = blockJ; j < Math.min(
48                         blockJ + blockSize, cols2); j++)
49                     {
50                         result[i][j] += firstMatrix[i][k
51                             ] * secondMatrix[k][j];
52                     }
53                 }
54             }
55         }
56     }
57 }
58 return result;
59 }
60 public static void main(String[] args) {
61     Scanner scanner = new Scanner(System.in);
62     int firstRows = readPositiveInt(scanner, "Введите количе
63     ство строк для первой матрицы: ");
64     int firstColumns = readPositiveInt(scanner, "Введите кол
65     ичество столбцов для первой матрицы: ");
66     int secondRows = readPositiveInt(scanner, "Введите колич
67     ество строк для второй матрицы: ");
68     int secondColumns = readPositiveInt(scanner, "Введите ко
69     личество столбцов для второй матрицы: ");
70     if (firstColumns != secondRows) {
71         System.out.println("Ошибка! Матрицы такой размерност
72             и нельзя перемножить! Столбцов в первой матрице д
73             олжно быть столько же, сколько строк во второй!")
74         ;
75         scanner.close();
76         return;
77     }
78     double[][] matrix1 = new double[firstRows][firstColumns
79         ];
80     double[][] matrix2 = new double[secondRows][
81         secondColumns];
82     for (int i = 0; i < firstRows; i++) {

```

```

66         for (int j = 0; j < firstColumns; j++) {
67             matrix1[i][j] = Math.random();
68         }
69     }
70     for (int i = 0; i < secondRows; i++) {
71         for (int j = 0; j < secondColumns; j++) {
72             matrix2[i][j] = Math.random();
73         }
74     }
75     int runs = 10;
76     long totalTime = 0;
77     for (int run = 0; run < runs; run++) {
78         long startTime = System.nanoTime();
79         double[][] result = multiply(matrix1, matrix2);
80         long endTime = System.nanoTime();
81         long duration = (endTime - startTime) / 1_000_000;
82         totalTime += duration;
83         System.out.println((run + 1) + ": " + duration + " м
            c");
84     }
85     System.out.println("Среднее время за 10 перемножений: "
        + (totalTime / runs) + " мс");
86     totalTime = 0;
87     for (int run = 0; run < runs; run++) {
88         long startTime = System.nanoTime();
89         double[][] result = multiplyOptimized(matrix1,
            matrix2);
90         long endTime = System.nanoTime();
91         long duration = (endTime - startTime) / 1_000_000;
92         totalTime += duration;
93         System.out.println((run + 1) + ": " + duration + " м
            c");
94     }
95     System.out.println("Среднее время за 10 оптимизированных
        перемножений: " + (totalTime / runs) + " мс");
96     scanner.close();
97 }
98 }

```

Повторно проведен анализ после исправления ошибки.

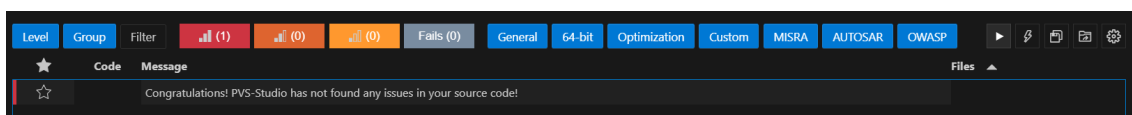


Рис. 5. Результат анализа

«Поздравляем! PVS-Studio не обнаружила никаких проблем в вашем

исходном коде!» - это сообщение подтверждает, что все выявленные ранее проблемы успешно исправлены.

Заключение

В рамках лабораторной работы был проведен статический анализ кода при помощи PVS-Studio, а также описана сама утилита. Использование PVS-Studio позволило обнаружить ошибку, связанную с неиспользуемой переменной. Этот недочет не был замечен при обычной инспекции кода — переменная `secondRows` в методе `multiply` объявлялась, но нигде не использовалась. Статический анализатор эффективно находит потенциальные ошибки.

После анализа кода был исправлен этот недочет. После внесенных исправлений повторный запуск PVS-Studio подтвердил отсутствие ошибок в коде.

Для меня выполнение данной работы стало ценным опытом. Во-первых, я познакомилась с инструментом статического анализа PVS-Studio, который ранее не использовала. Мне понравилась простота установки и интеграции с VS Code. Во-вторых, я убедилась, насколько полезным может быть автоматизированный поиск ошибок. Даже в таком небольшом проекте анализатор нашёл недочёт, который я упустила при ручном просмотре кода. Это подчёркивает важность статического анализа для повышения надёжности программ.